

HackNEX — Authentication & Authorization Design Document

Document Version: 1.0

Project: HackNEX

Organizer: NEXUS Club, Karunya Institute of Technology and Sciences

Event: October 7–9, 2026

Expected Participants: 1,500+

Authentication: Email + Password

Email Verification: Required

Social Login: Not supported

Admin Roles: Single Admin role

Frontend: React + Vite

Backend: Fastify + TypeScript

Database: PostgreSQL + Prisma

Email Service: Resend

1. Purpose

This document defines the complete authentication and authorization architecture for HackNEX.

It covers:

- Account creation
- Email verification
- Login
- Logout
- Session management
- Forgot password
- Password reset
- Authentication middleware
- Admin authentication
- Authorization
- Account security
- Rate limiting
- Token management
- Authentication states
- Error handling
- Security requirements
- Frontend authentication flow

- Backend authentication flow

The objective is to ensure that:

Only verified users can log in and register a HackNEX team, while only authorized administrators can access administrative functionality.

2. Authentication Philosophy

HackNEX will use a simple authentication system:

Email
+
Password
+
Email Verification
+
Secure Session

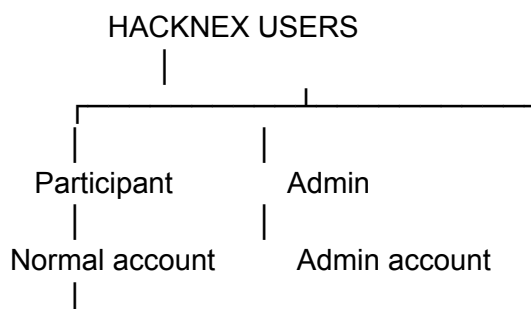
There will be **no Google/GitHub/social login**.

The system should prioritize:

- Security
 - Simplicity
 - Reliability
 - Good UX
 - Easy maintenance
 - Scalability for 1,500+ participants
-

3. User Types

HackNEX has two primary authenticated user categories.



Team Captain

Participant

A normal registered HackNEX account.

A participant can:

- Create an account
- Verify email
- Login
- Logout
- Reset password
- Create a team
- Register other 3 members
- View their registration
- Complete payment
- View registration status

Admin

An authorized HackNEX administrator.

Admins can:

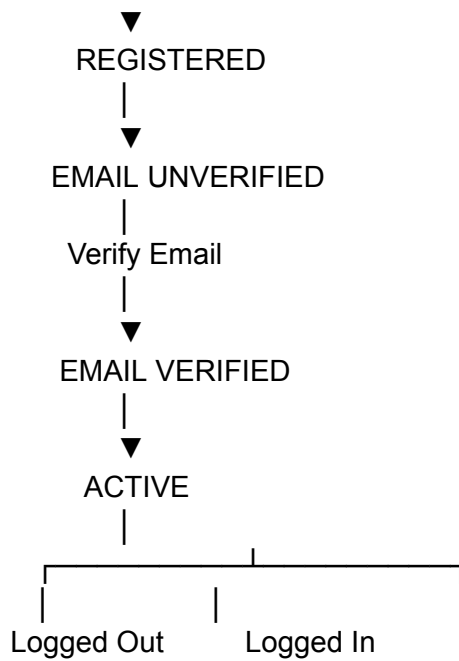
- Login
- View dashboard
- View teams
- View participants
- View payments
- Manage website content
- Manage media
- Manage FAQs
- Manage schedules
- Manage sponsors
- Manage announcements

There are **no different admin roles**.

4. Authentication States

A user can exist in several states.

ACCOUNT
|



Potential account status values:

UNVERIFIED
ACTIVE
SUSPENDED
DISABLED

5. Registration State

Account registration and HackNEX event registration are different concepts.

Account registration

Create HackNEX account



Verify email



Account active

Event registration

Login



Create team



Add 4 participants



Payment



Payment verified



HackNEX registration confirmed

This distinction is important.

A user can have an account without having registered for HackNEX.

6. Signup Flow

The complete signup flow:

User



Signup Page



Enter Name

Email

Password



POST /auth/signup



Validate Input



Check Existing Email



Hash Password



Create User



Generate Verification Token



Store Token



Send Verification Email



Show "Check your email"

7. Signup Fields

The account signup form contains:

Field	Required
Name	Yes
Email	Yes
Password	Yes
Confirm Password	Yes

Confirm Password is frontend validation and does not need to be stored in the database.

8. Email Normalization

Emails should be normalized before storage.

Example:

INFANT@Example.COM

becomes:

infant@example.com

The database should enforce:

User.email UNIQUE

This prevents multiple accounts using the same email.

9. Password Requirements

Passwords must satisfy a minimum security policy.

Recommended:

Minimum length: 8 characters

The frontend should provide password-strength feedback.

The backend remains the final validator.

Passwords must **never** be stored in plaintext.

10. Password Hashing

Recommended:

Argon2id

Flow:

Plain Password



Argon2id



Password Hash



PostgreSQL

The original password must never be stored.

11. Signup Duplicate Email

If the email already exists:

POST /auth/signup



Email exists



409 Conflict

Example:

```
{
  "success": false,
  "error": {
    "code": "EMAIL_ALREADY_EXISTS",
    "message": "An account with this email already exists."
  }
}
```

12. Email Verification

Email verification is mandatory.

After signup:

Account Created



Email Verification Token



Verification Email



User clicks link



Backend validates token



Email verified



Account activated

13. Verification Token

The system generates a cryptographically secure random token.

Example conceptual URL:

<https://hacknex.example/verify-email?token=TOKEN>

The token should:

- Be cryptographically random
 - Have a limited lifetime
 - Be single-use
 - Be invalidated after successful verification
-

14. Verification Token Expiry

Recommended:

24 hours

If the token expires:

Expired Token



Verification rejected



User requests new token

15. Resend Verification

Endpoint:

POST /api/v1/auth/resend-verification

Flow:

User



Enter Email



Backend



Check account



Generate new token



Invalidate previous token



Send new email

Rate limiting must